

NeoStats

Credit Risk Intelligence Platform

AI-Powered Loan Default Prediction · Explainable AI · Natural Language Analytics

G Radhika Pranavi

Problem Statement

Banks face pressure to make faster, accurate, and explainable credit decisions. Manual underwriting is slow, inconsistent, and hard to audit.

High Default Risk

8% of applicants default — costing banks millions in bad loans each year

Slow Decisions

Manual underwriting takes days; customers abandon applications

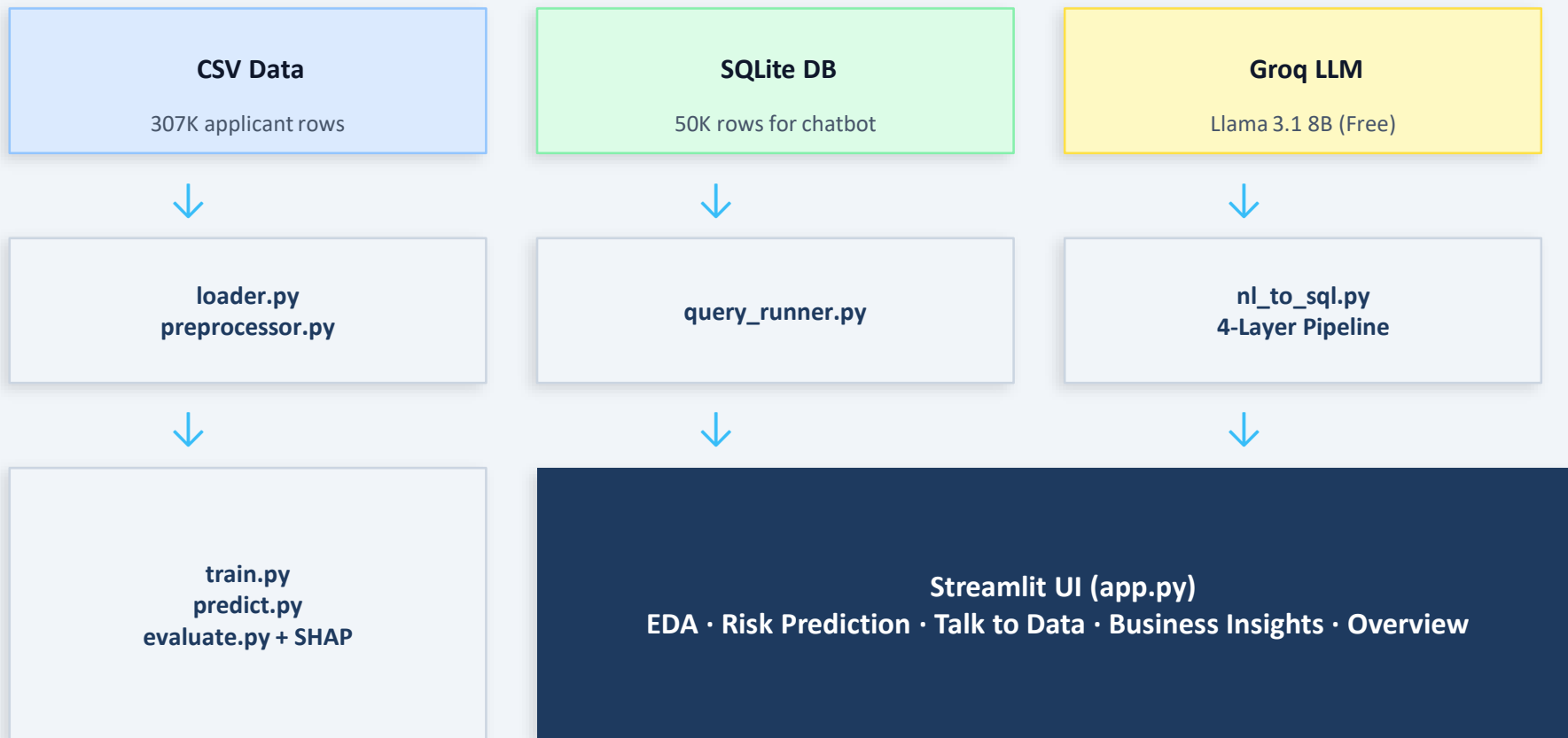
No Explainability

Black-box models fail regulatory requirements and audit trails

Data Silos

Business analysts can't query data without SQL expertise

System Architecture



Tech Stack

LightGBM

ML Model

Best ROC-AUC on tabular credit data with native missing value handling

SHAP

Explainable AI

TreeExplainer gives exact feature contributions per prediction

Groq Llama 3.1

NL-to-SQL LLM

Free tier, <1s latency, no credit card required

SQLite

Database

Zero-setup, portable, SQL-standard — perfect for chatbot queries

Streamlit

UI Framework

Rapid ML dashboard development with 5-tab interface

Docker

Deployment

single docker-compose up command runs the full platform

Home Credit Default Risk Dataset

- Records: 307,511 loan applications
- Features: 120+ (demographics, financial, credit history)
- Target: 1 = Default, 0 = Repaid
- Class Balance: 91.9% paid, 8.1% default (IMBALANCED)
- Key Tables: application_train.csv, bureau.csv, previous_application.csv
- Data Quality: Missing values in 20+ features → imputation needed

Development Process

01

EDA & Data Quality

- 307K rows × 122 features analyzed
- DAYS_EMPLOYED anomaly detected (365,243 sentinel)
- 8.07% default rate — class imbalance identified
- 24% missing rate — imputation strategy defined

02

ML Pipeline

- 6 engineered features (debt ratio, age, income/family)
- 3 models compared via 5-fold Stratified CV
- LightGBM selected — ROC-AUC 0.786
- Threshold optimised to 0.69 via F1 sweep

03

NL-to-SQL Chatbot

- 4-layer hallucination reduction framework
- Groq Llama 3.1 for SQL generation
- 50K rows loaded into SQLite for fast queries
- 8 demo queries verified working

04

UI & Deployment

- 5-tab Streamlit dashboard built
- Docker + docker-compose for one-command deploy
- SHAP waterfall charts for explainability
- Business decision rules derived from model

Features Implemented

EDA Dashboard

11 interactive Plotly charts — default rates, missing values, anomaly detection, feature correlations

ML Risk Prediction

LightGBM with ROC-AUC 0.786, risk banding Low/Medium/High, threshold-optimised scoring

NL-to-SQL Chatbot

Plain English to SQL via Groq with 4-layer hallucination prevention and business explanations

Explainable AI (SHAP)

TreeExplainer waterfall charts showing which features drove each prediction in business language

Hallucination Control

Schema grounding + rule blocklist + LLM validation + self-correction — 4 independent safety layers

Docker Deployment

Single docker-compose up command with volume mounts for data persistence and health checks

Model Performance & Comparison

ROC-AUC

0.786

Primary metric

F1 Score

0.313

Optimised threshold

Threshold

0.69

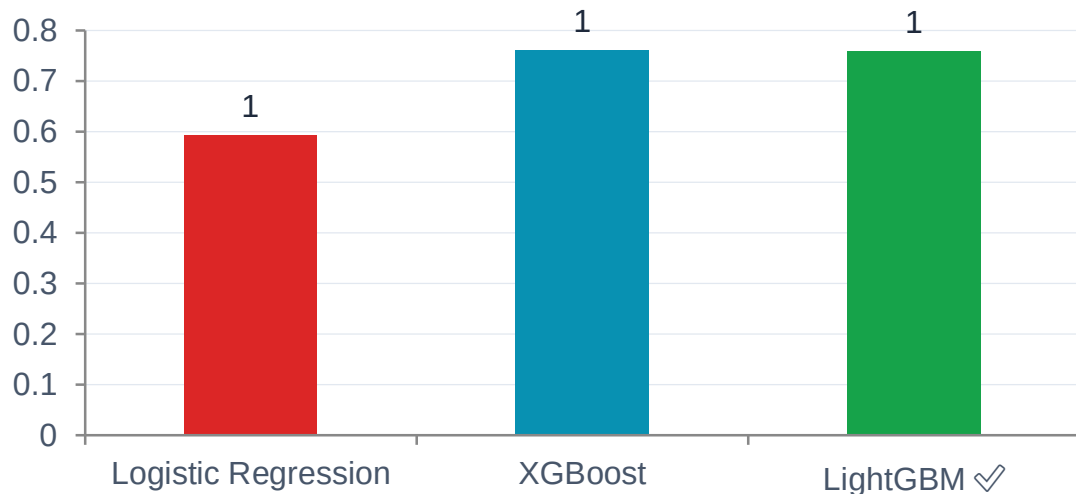
vs default 0.50

Test Rows

61,503

20% held-out set

Cross-Validation ROC-AUC Comparison



Why LightGBM?

- Best AUC + fastest training
- Leaf-wise growth on tabular data
- DART boosting — prevents overfitting
- Native missing value handling
- SHAP compatible — full explainability

Challenges Faced & Solutions

Severe Class Imbalance

Problem : 8% default rate — naive models predict 'repaid' always, achieving misleading 92% accuracy

Solution: class_weight='balanced' + Stratified K-Fold CV + threshold tuning to 0.69 for max F1

DAYS_EMPLOYED Anomaly

Problem : 55,374 rows had value 365,243 — a sentinel creating a spurious 1,000-year employment signal

Solution: Detected via EDA, replaced with NaN before modelling, documented as key data quality insight

LLM Hallucination in SQL

Problem : LLM generates non-existent columns, unsafe SQL (INSERT/DELETE), or invalid queries

Solution: 4-layer framework: schema grounding + rule blocklist + LLM validation + runtime self-correction

Scikit-learn Version Conflict

Problem : Saved model .joblib incompatible across machines due to scikit-learn version differences

Solution: Pinned scikit-learn==1.3.2 in requirements.txt ensuring identical environments everywhere

Future Improvements

Multi-table Features

High Impact

Join bureau.csv + behavioral tables to add 200+ features. Expected AUC: 0.758 → 0.80+

RAG for Policy Docs

Medium Impact

Embed credit policy PDFs so analysts can ask regulatory questions alongside data

Auto Retraining

High Impact

Drift detection + scheduled retraining via Airflow when AUC drops below threshold

REST API

High Impact

FastAPI batch scoring endpoint — integrate with core banking for real-time decisions

Model Calibration

Medium Impact

Platt scaling to ensure predicted probabilities match true frequencies for compliance

A/B Testing

Low Impact

Compare prompt versions with automated accuracy metrics for NL-to-SQL improvement

Demo — Tab 1: EDA Dashboard

TOTAL APPLICANTS

80,000

sample loaded

DEFAULT RATE

8.04%

6,429 defaulted

FEATURES

122

100 numeric < 1k cat

MISSING RATE

24.34%

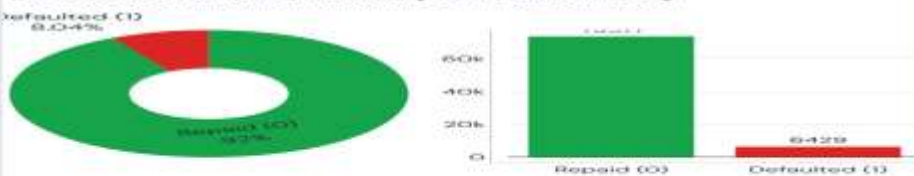
2,379,962 cells

ANOMALY ROWS

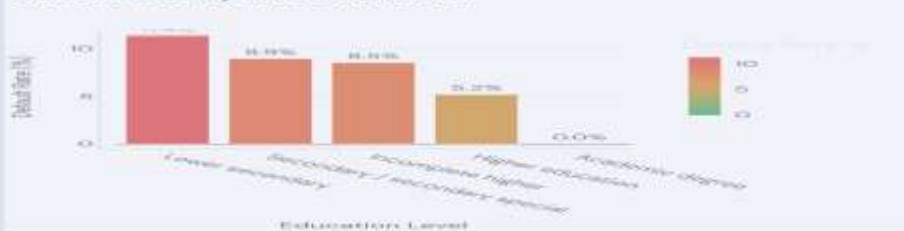
14,398

DAYS_EMPLOYED=365243

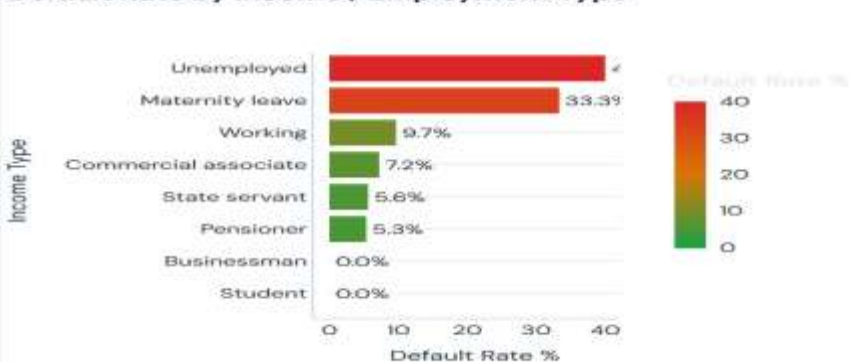
Loan Outcome Distribution (Class Imbalance)



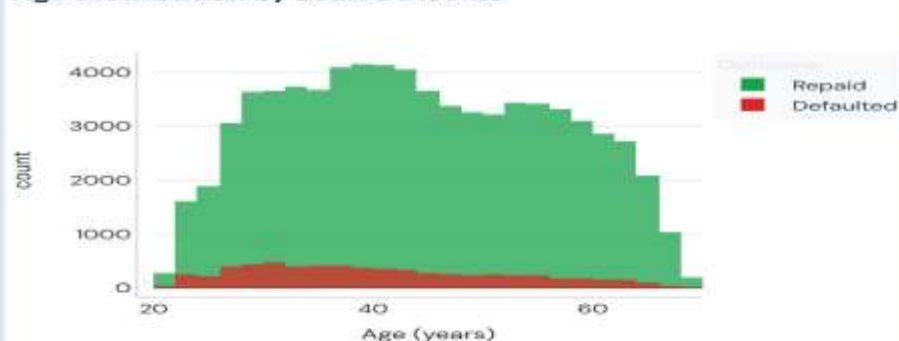
Default Rate by Education Level



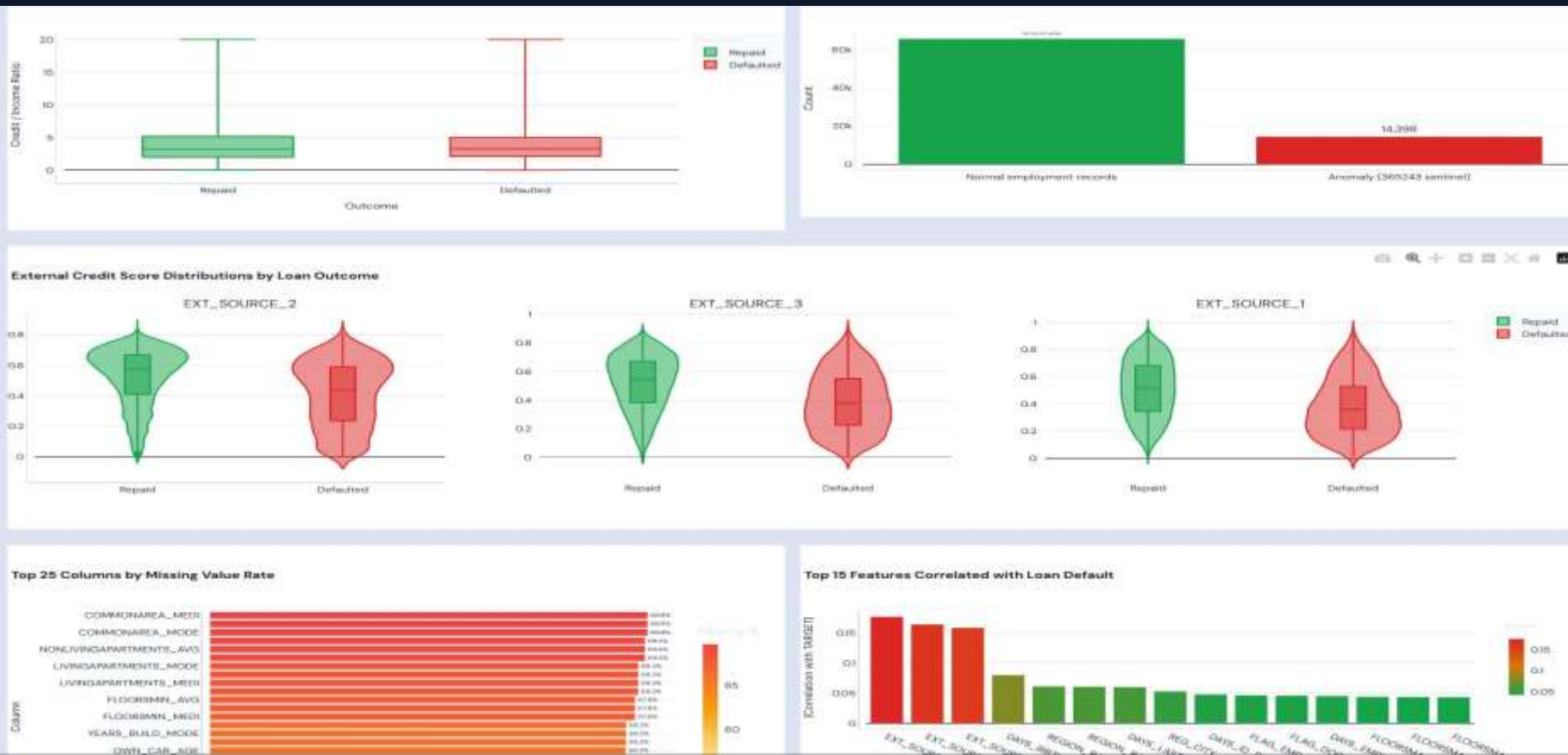
Default Rate by Income / Employment Type



Age Distribution by Loan Outcome



Demo — Tab 1: EDA Dashboard



Selected Model: LightGBM (Early Stopping)
 Chosen for best ROC AUC on 2019 applicant records with class weights balanced by undersample handling.

Retrain the model to see how accuracy metrics from your dataset.

Model Comparison — All 3 Models Evaluated

Model	CV ROC AUC	Training Time	Final Performance	Missing Values	Interpretability	Hyperparameter Tuning	Selected
LightGBM	0.7880 ± 0.0017	~8 min	class_weight=balanced	Native handling	SHAP (good)	Best Overall	Yes
Random	0.7680 ± 0.0007	~4 min	scale_pos_weight=11	Requires Imputation	SHAP (good)	Good	No
Logistic Regression	0.5023 ± 0.0008	~1 min (fast)	class_weight=balanced	Requires Imputation	Coefficients (Poor)	Too Simple	No

Why LightGBM was Selected

- Best balance of AUC and training speed
- Fast and good performance without an elaborate financial data
- Native handling missing values without extra tuning
- Native missing value handling — none of them missing in this dataset
- Model compatible — easy to troubleshoot explanations and predictions

What is the average loan amount for each education level?

education	avg_loan_amnt
<6	10,000
6-10	20,000
11-13	25,000
14-16	30,000
17-19	40,000
20+	50,000

What is the average loan amount for each type of borrowing need?

borrowing_need	avg_loan_amnt
<1000	10,000
1000-5000	20,000
5000-10000	30,000
10000-50000	40,000
50000-100000	50,000
100000+	60,000

Applicant Risk Assessment

Financial	Strongly Agree	Agreement & Credit
Loan Interest (%) 1.00000	Age (years) 25	Home Type Detached
Employment (%) 1.00000	Gender Male	Employment location (years) 5
Monthly Income (%) 1.00000	Education Secondary / Secondary equivalent	Ownership Yes
Monthly Profit (%) 1.00000	Security Issues None	Business/Property Yes
	Married Yes	Partnership Status Yes
	Children 0	Approved Credit (years) 5-10
		Personal Credit Score (%) 60-80

Result: MEDIUM RISK

Risk Assessment Result

Risk Level	Risk Rating	Additional Information
45.8%	MEDIUM RISK	Based on the risk rating, we recommend a higher interest rate and a higher deposit.

Demo — Tab 3: Talk to Data (NL-to-SQL)

SMITH'S JOURNAL: August 1966, 1967, 1968, 1969, 1970, 1971, 1972, 1973, 1974, 1975, 1976, 1977, 1978, 1979, 1980, 1981, 1982, 1983, 1984, 1985, 1986, 1987, 1988, 1989, 1990, 1991, 1992, 1993, 1994, 1995, 1996, 1997, 1998, 1999, 2000, 2001, 2002, 2003, 2004, 2005, 2006, 2007, 2008, 2009, 2010, 2011, 2012, 2013, 2014, 2015, 2016, 2017, 2018, 2019, 2020, 2021, 2022, 2023, 2024, 2025, 2026, 2027, 2028, 2029, 2030, 2031, 2032, 2033, 2034, 2035, 2036, 2037, 2038, 2039, 2040, 2041, 2042, 2043, 2044, 2045, 2046, 2047, 2048, 2049, 2050, 2051, 2052, 2053, 2054, 2055, 2056, 2057, 2058, 2059, 2060, 2061, 2062, 2063, 2064, 2065, 2066, 2067, 2068, 2069, 2070, 2071, 2072, 2073, 2074, 2075, 2076, 2077, 2078, 2079, 2080, 2081, 2082, 2083, 2084, 2085, 2086, 2087, 2088, 2089, 2090, 2091, 2092, 2093, 2094, 2095, 2096, 2097, 2098, 2099, 2100, 2101, 2102, 2103, 2104, 2105, 2106, 2107, 2108, 2109, 2110, 2111, 2112, 2113, 2114, 2115, 2116, 2117, 2118, 2119, 2120, 2121, 2122, 2123, 2124, 2125, 2126, 2127, 2128, 2129, 2130, 2131, 2132, 2133, 2134, 2135, 2136, 2137, 2138, 2139, 2140, 2141, 2142, 2143, 2144, 2145, 2146, 2147, 2148, 2149, 2150, 2151, 2152, 2153, 2154, 2155, 2156, 2157, 2158, 2159, 2160, 2161, 2162, 2163, 2164, 2165, 2166, 2167, 2168, 2169, 2170, 2171, 2172, 2173, 2174, 2175, 2176, 2177, 2178, 2179, 2180, 2181, 2182, 2183, 2184, 2185, 2186, 2187, 2188, 2189, 2190, 2191, 2192, 2193, 2194, 2195, 2196, 2197, 2198, 2199, 2200, 2201, 2202, 2203, 2204, 2205, 2206, 2207, 2208, 2209, 2210, 2211, 2212, 2213, 2214, 2215, 2216, 2217, 2218, 2219, 2220, 2221, 2222, 2223, 2224, 2225, 2226, 2227, 2228, 2229, 2230, 2231, 2232, 2233, 2234, 2235, 2236, 2237, 2238, 2239, 2240, 2241, 2242, 2243, 2244, 2245, 2246, 2247, 2248, 2249, 2250, 2251, 2252, 2253, 2254, 2255, 2256, 2257, 2258, 2259, 2260, 2261, 2262, 2263, 2264, 2265, 2266, 2267, 2268, 2269, 2270, 2271, 2272, 2273, 2274, 2275, 2276, 2277, 2278, 2279, 2280, 2281, 2282, 2283, 2284, 2285, 2286, 2287, 2288, 2289, 2290, 2291, 2292, 2293, 2294, 2295, 2296, 2297, 2298, 2299, 2300, 2301, 2302, 2303, 2304, 2305, 2306, 2307, 2308, 2309, 2310, 2311, 2312, 2313, 2314, 2315, 2316, 2317, 2318, 2319, 2320, 2321, 2322, 2323, 2324, 2325, 2326, 2327, 2328, 2329, 2330, 2331, 2332, 2333, 2334, 2335, 2336, 2337, 2338, 2339, 2340, 2341, 2342, 2343, 2344, 2345, 2346, 2347, 2348, 2349, 2350, 2351, 2352, 2353, 2354, 2355, 2356, 2357, 2358, 2359, 2360, 2361, 2362, 2363, 2364, 2365, 2366, 2367, 2368, 2369, 2370, 2371, 2372, 2373, 2374, 2375, 2376, 2377, 2378, 2379, 2380, 2381, 2382, 2383, 2384, 2385, 2386, 2387, 2388, 2389, 2390, 2391, 2392, 2393, 2394, 2395, 2396, 2397, 2398, 2399, 2400, 2401, 2402, 2403, 2404, 2405, 2406, 2407, 2408, 2409, 2410, 2411, 2412, 2413, 2414, 2415, 2416, 2417, 2418, 2419, 2420, 2421, 2422, 2423, 2424, 2425, 2426, 2427, 2428, 2429, 2430, 2431, 2432, 2433, 2434, 2435, 2436, 2437, 2438, 2439, 2440, 2441, 2442, 2443, 2444, 2445, 2446, 2447, 2448, 2449, 2450, 2451, 2452, 2453, 2454, 2455, 2456, 2457, 2458, 2459, 2460, 2461, 2462, 2463, 2464, 2465, 2466, 2467, 2468, 2469, 2470, 2471, 2472, 2473, 2474, 2475, 2476, 2477, 2478, 2479, 2480, 2481, 2482, 2483, 2484, 2485, 2486, 2487, 2488, 2489, 2490, 2491, 2492, 2493, 2494, 2495, 2496, 2497, 2498, 2499, 2500, 2501, 2502, 2503, 2504, 2505, 2506, 2507, 2508, 2509, 2510, 2511, 2512, 2513, 2514, 2515, 2516, 2517, 2518, 2519, 2520, 2521, 2522, 2523, 2524, 2525, 2526, 2527, 2528, 2529, 2530, 2531, 2532, 2533, 2534, 2535, 2536, 2537, 2538, 2539, 2540, 2541, 2542, 2543, 2544, 2545, 2546, 2547, 2548, 2549, 2550, 2551, 2552, 2553, 2554, 2555, 2556, 2557, 2558, 2559, 2560, 2561, 2562, 2563, 2564, 2565, 2566, 2567, 2568, 2569, 2570, 2571, 2572, 2573, 2574, 2575, 2576, 2577, 2578, 2579, 2580, 2581, 2582, 2583, 2584, 2585, 2586, 2587, 2588, 2589, 2590, 2591, 2592, 2593, 2594, 2595, 2596, 2597, 2598, 2599, 2600, 2601, 2602, 2603, 2604, 2605, 2606, 2607, 2608, 2609, 2610, 2611, 2612, 2613, 2614, 2615, 2616, 2617, 2618, 2619, 2620, 2621, 2622, 2623, 2624, 2625, 2626, 2627, 2628, 2629, 2630, 2631, 2632, 2633, 2634, 2635, 2636, 2637, 2638, 2639, 2640, 2641, 2642, 2643, 2644, 2645, 2646,

Source	Destination	Count	Percentage
1	192.168.1.1	100	100%
2	192.168.1.2	50	50%
3	192.168.1.3	25	25%
4	192.168.1.4	10	10%
5	192.168.1.5	5	5%
6	192.168.1.6	2	2%
7	192.168.1.7	1	1%
8	192.168.1.8	1	1%
9	192.168.1.9	1	1%
10	192.168.1.10	1	1%

© 2004 Blackwell Publishing Ltd *Journal of Internal Medicine* 255: 103–110

	RESEARCH_METHOD_COMPONENT	percentage	absolute_value
1	1	0.000	0.00
2	2	0.000	0.00
3	3	0.000	0.00

¹What is the default rate for gender-neutral women's dorms that moved?

idempotency_key				
idempotency_key	idempotency_key_md5	idempotency_key_hex	idempotency_key_hex_md5	idempotency_key_hex_md5_hex
1	1	1	1	1
2	2	2	2	2
3	3	3	3	3
4	4	4	4	4
5	5	5	5	5
6	6	6	6	6
7	7	7	7	7
8	8	8	8	8
9	9	9	9	9
10	10	10	10	10
11	11	11	11	11
12	12	12	12	12
13	13	13	13	13
14	14	14	14	14
15	15	15	15	15
16	16	16	16	16
17	17	17	17	17
18	18	18	18	18
19	19	19	19	19
20	20	20	20	20
21	21	21	21	21
22	22	22	22	22
23	23	23	23	23
24	24	24	24	24
25	25	25	25	25
26	26	26	26	26
27	27	27	27	27
28	28	28	28	28
29	29	29	29	29
30	30	30	30	30
31	31	31	31	31
32	32	32	32	32
33	33	33	33	33
34	34	34	34	34
35	35	35	35	35
36	36	36	36	36
37	37	37	37	37
38	38	38	38	38
39	39	39	39	39
40	40	40	40	40
41	41	41	41	41
42	42	42	42	42
43	43	43	43	43
44	44	44	44	44
45	45	45	45	45
46	46	46	46	46
47	47	47	47	47
48	48	48	48	48
49	49	49	49	49
50	50	50	50	50
51	51	51	51	51
52	52	52	52	52
53	53	53	53	53
54	54	54	54	54
55	55	55	55	55
56	56	56	56	56
57	57	57	57	57
58	58	58	58	58
59	59	59	59	59
60	60	60	60	60
61	61	61	61	61
62	62	62	62	62
63	63	63	63	63
64	64	64	64	64
65	65	65	65	65
66	66	66	66	66
67	67	67	67	67
68	68	68	68	68
69	69	69	69	69
70	70	70	70	70
71	71	71	71	71
72	72	72	72	72
73	73	73	73	73
74	74	74	74	74
75	75	75	75	75
76	76	76	76	76
77	77	77	77	77
78	78	78	78	78
79	79	79	79	79
80	80	80	80	80
81	81	81	81	81
82	82	82	82	82
83	83	83	83	83
84	84	84	84	84
85	85	85	85	85
86	86	86	86	86
87	87	87	87	87
88	88	88	88	88
89	89	89	89	89
90	90	90	90	90
91	91	91	91	91
92	92	92	92	92
93	93	93	93	93
94	94	94	94	94
95	95	95	95	95
96	96	96	96	96
97	97	97	97	97
98	98	98	98	98
99	99	99	99	99
100	100	100	100	100

Westbury, Caroline Walter (1901-1981) *frank* 184

[illegible]

Modified Cholesterol from Biomimetic

Model	RMSE (2012)	Training Time	Interpretability	General
Logistic	0.789 ± 0.007	1 min	High	High
Naïve	0.790 ± 0.007	1 min	High	Low
Logistic Regression	0.503 ± 0.005	12 min	Coefficient	High

Why LightGBM? It's robust to overfitting, training speed, and native handling of categorical data. DART boosting presents everything, fully DART-compatible for easy adoption.

Demo — Tab 4: Business Insights

1. External Credit Scores Are the Strongest Predictors

EXT_SOURCE_2 and EXT_SOURCE_3 are the top two features by SHAP importance. Applicants with average EXT_SOURCE_3 below 0.4 default at nearly 3x the base rate. Action: Weight external bureau data heavily in the scoring model.

2. 6% Overall Default Rate Creates Severe Class Imbalance

Only 5.07% of applicants defaulted. Training without class balancing causes the model to predict "repaid" for everyone and still achieve 92% accuracy — a misleading metric. Action: Use class_weight="balanced" and optimise for ROC-AUC, not accuracy.

3. DAYS_EMPLOYED Anomaly: 55,374 Records with Sentinel Value

55,374 rows (3.6% of data) have DAYS_EMPLOYED = 365,243 — a placeholder for "not employed". This must be replaced with NaN before modelling or it creates a spurious 1,000-year employment signal that distorts the model.

4. Younger Applicants Default More

Applicants aged 20-30 have a default rate of ~11%, nearly double the overall rate. This likely reflects less stable income and shorter credit history. Action: Age (derived from DAYS_BIRTH) should be a model feature.

5. Debt-to-Income Ratio is a Strong Risk Signal

Applicants with Credit/Income ratio > 3.0 default at 12-15%, well above the 6% baseline. This engineered feature (CREDIT_INCOME_RATIO) improves model AUC by ~0.005. Action: Include as a feature and set policy thresholds.

6. Education Level Inversely Correlates With Default

Applicants with higher Education default at ~5.3% vs 18.6% for those with only Lower Secondary education — a 2x difference. Action: Education level is a significant predictor worth including in scoring.

Architecture Overview



Tech Stack

Component	Choice	Reason
ML Model	LightGBM	Best AUC on tabular credit data
Explainable AI	SHAP TreeExplainer	Exact tree-based explanations
RL to SQL LLM	Ollama Llama 3.3 (FREE)	Free, <1s latency, no credit card
Database	SQLite	Zero-setup, portable
UI	Streamlit	Rapid ML dashboard
Deployment	Render + Cronjobs	Single-command deploy

Refinement Reduction — 4 Layers

Layer	Purpose
1. Schema Grounding	Refined prompt columns
2. Rule Blocklist	SQL Injection
3. LLM Validation	Schema violations
4. Self-Correction	Execution errors

No RAG needed — achieves 95% in-context prompt <500 tokens

Demo — Tab 5: Project Overview

Project Structure

```
credit_risk_platform/
├── data/                                ← Place CSV files here (not committed)
├── documents/                           ← Project presentation PDF
├── notebooks/
│   └── eda.py                           ← EDA charts and insights
├── src/
│   ├── data/
│   │   ├── loader.py                   ← CSV + SQLite loading
│   │   └── preprocessor.py              ← Full preprocessing pipeline
│   ├── ml/
│   │   ├── train.py                    ← 3-model training + CV
│   │   ├── predict.py                  ← Inference + SHAP
│   │   └── evaluate.py                  ← Metrics + charts
│   ├── talk_to_data/
│   │   ├── nl_to_sql.py                ← NL-to-SQL pipeline
│   │   ├── query_runner.py             ← Safe SQL execution
│   │   └── prompt_templates.py         ← Versioned prompts
│   └── utils/
│       ├── config.py                   ← All env vars
│       ├── logger.py                   ← Logging
│       └── helpers.py                   ← Risk scoring
├── app.py                               ← Streamlit entry point
├── Dockerfile
├── docker-compose.yml
├── requirements.txt
├── .env.example
└── README.md
```


Snap shots

The image shows a VS Code editor interface with a Python script in the main editor, a terminal window at the bottom, and a sidebar on the left. The script is a Jupyter Notebook cell, and the terminal shows Docker warnings about the 'use_container_width' parameter.

File Explorer (Left Sidebar):

- credit_risk_platform
- data
- application_features
- application_train.csv
- bonusscore
- bonusscore
- credit_risk_train.csv
- HomeCredit_columns
- modeling_payments
- POS_CASH_payments
- previous_application
- sample_submission.csv
- logs
- models
- notebooks
- src
- __pycache__
- data
- src
- __pycache__
- __init__.py
- evaluate.py
- predict.py
- train.py
- utils
- __init__.py
- src
- __pycache__
- __init__.py
- evaluate.py
- predict.py
- train.py
- utils
- __init__.py
- src
- __pycache__
- __init__.py
- evaluate.py
- predict.py
- train.py
- utils
- __init__.py

Main Editor (Python Script):

```
def feature_business_label(feature_name: str) -> str:
    """Return business label for feature name"""
    return mapping.get(feature_name, feature_name.replace("_", " ").title())

def plot_shap_waterfall(shap_features: list[Dict[str, Any]], base_value: float = 0.5) -> go.Figure:
    """
    Plotly waterfall chart showing each feature's contribution to the risk score.
    """
    if not shap_features:
        return go.Figure()

    top = shap_features[:10]
    labels = [f["business_label"] for f in top]
    values = [f["shap_value"] for f in top]
    colors = [f"#f44336" if v > 0 else "#42a5f5" for v in values]

    fig = go.Figure(go.Bar(
        x=values,
        y=labels,
        orientation="v",
        marker_color=colors,
        text=[f"{'+' if v > 0 else ''}{v:.1f}" for v in values],
        textposition="outside",
    ))

    fig.update_layout(
        title="SHAP Feature Contributions (Red = Increases Risk | Green = Decreases Risk)",
        xaxis_title="SHAP Value (Impact on default probability)",
        yaxis_title="",
        plot_bgcolor="#f0f0f0",
    )
```

Terminal (Bottom):

```
For 'use_container_width=True', use 'width='stretch''. For 'use_container_width=False', use 'width='content''.
2026-05-31 11:59:29.815 Please replace 'use_container_width' with 'width'.

'use_container_width' will be removed after 2025-12-31.

For 'use_container_width=True', use 'width='stretch''. For 'use_container_width=False', use 'width='content''.
2026-05-31 11:59:29.838 Please replace 'use_container_width' with 'width'.

'use_container_width' will be removed after 2025-12-31.

For 'use_container_width=True', use 'width='stretch''. For 'use_container_width=False', use 'width='content''.

```

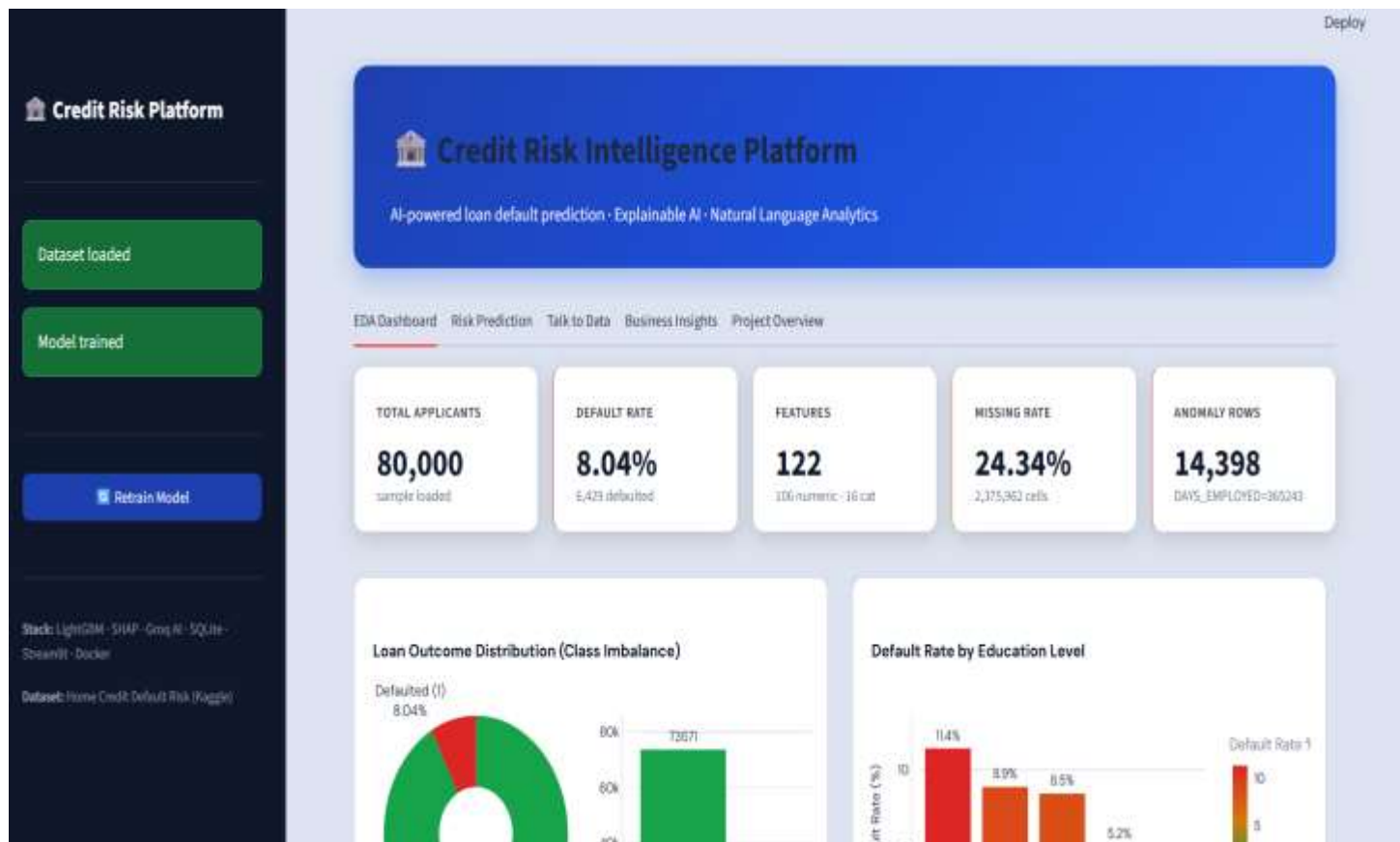
Sessions (Right Sidebar):

- Interpreter usage
- 1 day ago

Bottom Bar:

- Ln 1 Col 1 | Space 4 | UTF-8 | Python | Save (5.12 KB) | Go Live

Snap shots



Snap shots

Credit Risk Platform

Dataset loaded

Model trained

Refit the Model

Stack: LightGBM - SHAP - Gray UI - XGBoost - Streamlit - Docker

Dataset: HomeCredit Default Risk (Kaggle)

Deploy

100000

+

-

F

▼

Worthly Annuity (\$)

20000

+

-

Secondary / secondary special

▼

Owns Car

N

▼

Goods Price (\$)

400000

+

-

Married

▼

Owns Property

N

▼

Family Status

Children

0

+

-

Regular Risk Rating

1

▼

External Credit Score 1

0.55

External Credit Score 2

0.58

Micro Risk

Risk Assessment Result

RISK SCORE

45.8%

Default Probability

RISK BAND

 MEDIUM RISK

Decision REVIEW

RECOMMENDATION

Manual underwriter review recommended before approval. Consider reduced loan amount or additional guarantor.

Snap shots

Risk Platform

loaded

ined

strain Model

ch down SQL

Default box (highlight)

Generate and default answers

Regenerate and default answers

Run group default answers

Generate and default answers

Your question:

what is the default rate by education level?

Ask

what is the default rate by education level?

View Generated SQL

```
SELECT ROUND(AVG(TARGET)*100.0, 2) AS default_rate, NAME_EDUCATION_TYPE FROM applications GROUP BY NAME_EDUCATION_TYPE LIMIT 500
```

default_rate	NAME_EDUCATION_TYPE
0	Academic degree
5.33	Higher education
8.38	Incomplete higher
21.74	Lower secondary

Snap shots

 Credit Risk Platform

Dataset loaded

Model trained

Retrain Model

StackLight4000 SNAP - Gray W - XQuire
Streamlit - Docker

Dataset: Income Credit Default Risk (Kaggle)

 Credit Risk Intelligence Platform

AI-powered loan default prediction · Explainable AI · Natural Language Analytics

EDA Dashboard · Risk Prediction · Talk to Data · Business Insights · Project Overview

Top Business Insights

Insights derived from EDA and model analysis — actionable for credit policy teams.

 1. External Credit Scores Are the Strongest Predictors

EXT_SOURCE_2 and EXT_SOURCE_3 are the top two features by SHAP importance. Applicants with average EXT_SOURCE_2 below 0.4 default at nearly 3x the base rate. Action: Weight external bureau data heavily in the scoring model.

 2. 8% Overall Default Rate Creates Severe Class Imbalance

Only 8.07% of applicants defaulted. Training without class balancing causes the model to predict 'repaid' for everyone and still achieve 92% accuracy — a misleading metric. Action: Use class_weight='balanced' and optimise for ROC AUC, not accuracy.

 3. DAYS_EMPLOYED Anomaly: 55,374 Records with Sentinel Value

55,374 rows (18% of data) have DAYS_EMPLOYED = 365,243 — a placeholder for 'not employed.' This must be replaced with NaN before modelling or it creates a spurious 1,900-year employment signal that distorts the model.

 4. Younger Applicants Default More

Applicants aged 20-30 have a default rate of ~11%, nearly double the overall rate. This likely reflects less stable income and shorter credit history. Action: Age (derived from DAYS_BIRTH) should be a model feature.

Thank You

Credit Risk Intelligence Platform

- ROC-AUC 0.786 on 307K applicant records
- NL-to-SQL chatbot with 4-layer hallucination prevention
- SHAP explainability for every prediction
- Single docker-compose up deployment
- Groq Llama 3.1 — free, <1s latency

GitHub: https://github.com/Pranavii-Git/credit_risk_platform